

LASTENHEFT

Freigab Cockpit Next

Anforderungsspezifikation — HVS Software-Freigabe-Cockpit

Version	1.0
Datum	10. Juni 2026
Status	Final
Herausgeber	BMW Group · P3 Group

Vertraulich — nur für den internen Gebrauch

Inhaltsverzeichnis

1. Zweck, Kontext & Zielsetzung

- [1.1 Ausgangslage](#)
- [1.2 Lösungsidee](#)
- [1.3 Geschäftsziele \(messbar / qualitativ\)](#)
- [1.4 Automatisierungs-Zielbild](#)

2. Fachliches Glossar & Datenmodell

- [2.1 Begriffe](#)
- [2.2 Kern-Geschäftslogik \(verbindlich\)](#)

3. Daten- & Integrationsarchitektur

- [3.1 Dataverse-Tabellen \(System of Record\)](#)
- [3.2 Datenfluss & Build-Strategie](#)
- [3.3 ETL-Dataflow „FGC_DATA“ \(Power Query Online → Dataverse\)](#)

4. Funktionale Anforderungen

- [4.0 App-Shell & Navigation](#)
- [4.1 Freigabe Timeline \(Gantt-Hauptansicht\) — \[/freigabe-timeline\]\(#\)](#)
- [4.2 Cockpit \(aktionsorientierte Kartenansicht\) — \[/cockpit\]\(#\)](#)
- [4.3 Verschränkungen — \[/verschraenkungen\]\(#\)](#)
- [4.4 Dataverse Admin — \[/dataverse\]\(#\)](#)
- [4.5 WMM-Verwaltung \(derzeit simuliert\)](#)

5. Nicht-funktionale Anforderungen

6. Abgrenzung (nicht im aktuellen Stand enthalten)

7. Offene Punkte & Roadmap

8. Rollen (vorgesehen)

9. Kundenanforderungen & Umsetzungsstand

- [9.0 Allgemein](#)
- [9.1 WMM](#)
- [9.2 IP3](#)
- [9.3 DEEP](#)
- [9.4 Jira](#)
- [9.5 MIA](#)

10. Datenmodell (ER-Diagramm)

- [10.1 Entitäten \(Auswahl der Schlüsselattribute\)](#)
- [10.2 Beziehungen](#)
- [10.3 Mermaid-Quelltext \(zur Weiterverwendung\)](#)

11. Zielbild & Systemlandschaft (Gesamtscope)

- [11.1 Prozess- & Systemkontext](#)
- [11.2 Funktionssäulen](#)
- [11.3 Quellsysteme](#)
- [11.4 Zielsysteme](#)
- [11.5 Schlüsselkonzepte & Berechnungslogiken](#)
- [11.6 Epic- & Feature-Backlog \(E0–E7\) mit Umsetzungsstand](#)

Dokumentinformationen

Feld	Wert
Dokumenttyp	Lastenheft (Anforderungsspezifikation)
Projekt	Freigabcockpit Next — HVS Software-Freigabe-Cockpit (P3 Group)
Anwendung	Microsoft Power Apps Code App (React, TypeScript)
Plattform / Datenhaltung	Microsoft Dataverse
App-ID	d913d290-2c06-455d-b1d3-71b2f9bb4cbf
Umgebung (Produktion)	de4c7f26-7fe2-ef12-8c20-d939f27f1356
Dataverse-Connection	shared_commondataserviceforapps
Version	1.0
Datum	10.06.2026
Status	Final
Klassifizierung	Vertraulich – nur für den internen Gebrauch

1. Zweck, Kontext & Zielsetzung

1.1 Ausgangslage

Die Freigabe von BMW-Hochvolt-Speicher-Software (HVS) wird heute in einer manuell gepflegten Excel-Datei (`Gen6_EM_Freigabecockpit_Offline.xlsx`) verfolgt. Diese Datei wird wöchentlich von Hand aktualisiert, ist nicht mehrbenutzerfähig, fehleranfällig und entkoppelt von den Quellsystemen (Jira, CDH/IP3, DEEP, WMM). Der Freigabestand je Speicher, Kalenderwoche und Softwarestand lässt sich darin nur mühsam und ohne Konsistenzgarantie ablesen.

1.2 Lösungsidee

Das **Freigabecockpit Next** ersetzt diese Excel-Datei durch eine in Microsoft Dataverse verankerte Webanwendung. Die App liest die Stammdaten (I-Stufen, HVS, Penthouse-/Verbund-Tickets, CDH-Meilensteine) aus Dataverse-Tabellen, die ihrerseits aus den Quellsystemen befüllt werden, und stellt den Soll-/Ist-Stand der Freigaben entlang der Kalenderwochen in zwei sich ergänzenden Ansichten dar:

- einer **planungsnahen Gantt-Timeline** (alle Speicher x Wochen in einem Raster), und
- einem **aktionsorientierten Cockpit** (eine Karte je Speicher, schnelles Setzen von Freigaben).

1.3 Geschäftsziele (messbar / qualitativ)

1. **Eine gemeinsame Wahrheit** statt verteilter Excel-Stände — alle Beteiligten sehen denselben Datenstand.
2. **Wöchentlicher Überblick auf einen Blick:** Welche Softwarestände (I-Stufen) sind je Speicher in welcher KW aktiv, welches Freigabe-Level ist geplant (Soll) und welches tatsächlich erreicht (Ist)?
3. **Aufmerksamkeitssteuerung:** offene WMM-Deltas, fehlende Sachnummern und anstehende Tickets werden proaktiv als Kennzahlen hervorgehoben, damit nichts durchrutscht.
4. **Effiziente Pflege:** Bulk-Operationen (mehrere Speicher x mehrere Wochen in einem Schritt) ersetzen die zellenweise Excel-Arbeit; Verschränkungs- und Kaskadenlogik verhindert inkonsistente Einzeleinträge.
5. **Nachvollziehbarkeit:** Verschiebungen, manuelle Einträge und Verschränkungen sind sichtbar markiert und umkehrbar.

1.4 Automatisierungs-Zielbild

Im Gegensatz zur manuellen Pflege des heutigen Excel-Dokuments soll FreigabecockpitNEXT Informationen über **automatisierte Schnittstellen aus mehreren Quellsystemen** abgreifen, im User-Interface die **Freigabetermine für Hochvolt-Speicher bestätigen** lassen und auf Basis der gesammelten Daten anschließend **Folgesysteme befüllen** (Jira CodeCraft, MIA, Freigabemplate/SharePoint, FUSI Confluence). Das vollständige Zielbild mit allen Quell- und Zielsystemen, Funktionssäulen, Rollen und dem Epic-Backlog ist in **§11** mit einer Prozess-Skizze (Abb. 2) beschrieben; der aktuelle Umsetzungsstand ist in **§4** (Funktionen) und **§9** (Anforderungsbewertung) dokumentiert.

2. Fachliches Glossar & Datenmodell

2.1 Begriffe

Begriff	Bedeutung & Detail
HVS	Hochvoltspeicher / Speichermuster. Eine HVS-Zeile = Kombination aus Speichertyp, WBS-Typ und Muster. Identifiziert über Composite-Key <code>speicher-wbsType</code> (z. B. <code>P-114758-VS_I</code>).
Speicher	HVS-Kurzcode, abgeleitet aus dem Speichertyp-String (erstes Token), z. B. <code>B6R00</code> . Die zugehörige P-Nummer ist z. B. <code>P-114758</code> .
WBS-Typ	Klassifikation des Speichers, z. B. <code>VS_I</code> .
Muster	Prototypen-/Bauphase, z. B. <code>D1.0</code> , <code>C0.8</code> .
Penthouse (PTH)	Software-Dachpaket; auch der Name des zugehörigen Jira-Tickets. HVS-Feld <code>penthouse</code> .
I-Stufe / Softwarestand	Composite-Key <code>SE_TERMIN-REIFE</code> , z. B. <code>26-07-480</code> . Ein konkreter Software-Reifestand eines Lieferzyklus.
SE_TERMIN	Software-Lieferzyklus im Format <code>yy-ww</code> , z. B. <code>26-07</code> , <code>26-11</code> , <code>27-03</code> . Treibt Farbe und Default-Sichtbarkeit.
REIFE / Reifegrad	Ganzzahliger Reifegrad, z. B. 460, 480, 500. 500 = Serienreife (höchste Reife).
ATS	*Anfangs-Termin Software* (Teststart). Beginn des Freigabefensters.
SAB	*Softwareabgabe* (Liefertermin) $\approx$ ATS + 8 Wochen.
OFFSET	Position innerhalb des Fensters: <code>ATS WEEK</code> (= Woche 0), <code>ATS+1 ... ATS+7</code> , <code>SAB WEEK</code> (= Woche 8).
Softwarestand-Leiter	Erweiterte 25-stufige Skala für Verschiebungen: <code>ATS-8 ... ATS-1, ATS, ATS+1 ... ATS+7, SAB (= ATS+8), SAB+1 ... SAB+8</code> . Intern als Index 0–24 abgebildet (ATS = 8, SAB = 16).
Freigabe-Level	Eskalationsstufen: <code>X</code> , <code>RSTB</code> , <code>L1</code> , <code>L2</code> , <code>L3</code> , <code>L4</code> . Im vollen Datenmodell zusätzlich „Erstfreigabe“/„geplant“/„freigegeben“-Varianten mit Rang 1–22 (<code>LEVEL_RANK</code>).
Verbund	Verbundfreigabe (Swap-Ticket), typischerweise zum Do-Termin.
WMM	Werkzeug-/Sachnummern-Management. Liefert ZSMB-Verknüpfung und Sachnummern (Quelle: BATKAS/IHP).
ZSMB	Zusammenbau-Identifizier im WMM, z. B. <code>ZSMB-114758-VS_I-D1.0</code> .
Sachnummer	BMW-Teilenummer, Format <code>83 21 7 ### ##</code> , mit Typ („Kackel“-Position) und Beschreibung.
CDH / IP3	Langfrist-HVS-Freigabetermine je Muster-Phase (Soll-Meilensteine).
Verschränkung (Entanglement)	Bindung „Softwarestand $\rightarrow$ Speicher“. Nach dem Binden gelten alle künftigen Freigaben dieses Softwarestands ausschließlich für den gebundenen Speicher.
MIA	Technische Bewertung (5 Stufen RSTB/L1–L4). Im UI als Badge-Platzhalter vorgesehen, Funktion in Phase 3.
BRV	Fahrzeug-Referenz, aktuell ausschließlich <code>NA05</code> (über Power-Query-Filter hartkodiert).

2.2 Kern-Geschäftslogik (verbindlich)

- **Aktiv-Zeitraum:** Eine I-Stufe ist von ihrer **ATS-Woche bis einschließlich SAB-Woche** aktiv. Mehrere I-Stufen können gleichzeitig aktiv sein (je SE_TERMIN bzw. Reifegrad).
- **Wochen-Offset:** Die Position innerhalb des Fensters bestimmt das Offset-Label (`ATS WEEK` ... `SAB WEEK`) und wird je I-Stufe aus der exploded Tabelle `cr9b2_ilevels_1` gelesen (eine Zeile je I-Stufe x Offset).
- **„Aktiver Softwarestand“ einer Zelle:** In der Cockpit-Kartenansicht wird je KW der Softwarestand mit dem **höchsten Reifegrad** gewählt, dessen SE_TERMIN sichtbar (nicht ausgefiltert) ist; Gleichstand wird lexikografisch über den I-Stufen-Key gebrochen.
- **Kalenderwochen:** Durchgängig **ISO-Wochen, Montag-basiert**; Jahresgrenzen werden über das ISO-Wochenjahr korrekt behandelt. Schlüsselformat intern: `"YYYY-ww"` (z. B. `2026-07`).
- **BRV-Bereinigung:** Beim Auflösen von Ticket-I-Stufen wird ein führender BRV-Präfix entfernt (`NA05-26-07-510` $\rightarrow$ `26-07-510`); dieselbe I-Stufe über mehrere BRVs kollabiert zu einem Key.

3. Daten- & Integrationsarchitektur

3.1 Dataverse-Tabellen (System of Record)

Tabelle	Inhalt	Snapshot-Rows*
cr9b2_brv	Fahrzeug-Referenzen (BRV; aktuell NA05)	5
cr9b2_cdh_ip3	CDH/IP3 Meilenstein-/Freigabetermine je Muster-Phase	14
cr9b2_hvs	Hochvoltspeicher-Stammdaten (Speichertyp, WBS, Muster, Penthouse)	6
cr9b2_ilevels_1	I-Stufen „exploded“: eine Zeile je I-Stufe x Offset-Woche	965
cr9b2_ip3_freigaben	IP3-Freigaben	13
cr9b2_planned_component_approvals	Penthouse-Tickets (aus Jira synchronisiert)	77
cr9b2_verbundfreigaben	Verbund-/Swap-Tickets	898
cr9b2_wbs_type_mapping	WBS-Typ-Mapping	7

* Zeilenanzahl gemäß aktuellem Datenbestand (Referenz-Snapshot).

3.2 Datenfluss & Build-Strategie

- Quellsysteme → ETL → Dataverse:** Jira, CDH/IP3, DEEP und WMM liefern Rohdaten. Diese werden über den **Power-Apps-Dataflow „FGC_DATA“** (Power Query Online, Bereich *Dataintegration*) transformiert und in die **cr9b2_***-Tabellen geladen — die komplette Bereinigung, Filterung und Anreicherung passiert **vor** dem Upload in Dataverse (siehe ausführlich §3.3). Die App betrachtet **planned_component_approvals** und **verbundfreigaben** als **read-only Jira-Sync** und schreibt nie zurück.
- Dataverse → Datenstand der App:** Ein Aufbereitungsprozess liest die Dataverse-Tabellen aus und erzeugt einen normalisierten, im App-Paket mitgelieferten Datenstand. Laufzeit ~2–3 Minuten.
- Datenstand → Anwendungsmodell:** Das Datenmodul der Anwendung bildet die ursprüngliche Aufbereitungslogik nach (z. B. Ableitung des HVS-Kurzcodes, Normalisierung **ATS** → **ATS WEEK**, Sortierung der Offsets) und stellt typisierte Datenstrukturen (Softwarestand, HVS, Penthouse-/Verbund-Ticket) bereit.
- Live-CRUD im Betrieb:** Ein Dataverse-Zugriffsdienst stellt Lese- und Schreiboperationen (Auflisten/Anlegen/Ändern/Löschen je Tabelle) sowie ein Neu-Einlesen des Datenstands bereit. Hierauf setzen die Dataverse-Admin-Seite und der „Aktualisieren“-Button auf.
- Laufzeit-Persistenz im Browser:** WMM-Datensätze, Verschränkungen und Verschiebungen geplanter Freigaben werden derzeit im lokalen Browser-Speicher gehalten und tab-übergreifend synchronisiert. Die Migration in eigene Dataverse-Tabellen ist vorgesehen (§7).

3.3 ETL-Dataflow „FGC_DATA“ (Power Query Online → Dataverse)

Der vorgelagerte **Standard-Dataflow „FGC_DATA“** (Power Apps · *Dataintegration*, Umgebung **b83cf532-5520-e124-ad5e-012e1e43d44d**) übernimmt die **gesamte ETL-Verarbeitung**, bevor Daten nach Dataverse geschrieben werden. Er besteht aus fünf Power-Query-(M-)Abfragen, deren Verarbeitungsschritte nachfolgend dokumentiert sind. Referenz-Datenstand: Snapshot **20260420_1641**.

Datenquellen (Extract): Jede Abfrage liest eine **CSV-Datei aus SharePoint** (Bibliothek **.../Microsoft Power Query/Uploaded Files/**). Diese CSVs sind manuell hochgeladene Exporte aus den Quellsystemen (Jira-Export der Tickets, iLevel-/Verbund-Listen). Konvention im Dateinamen: Zeitstempel-Präfix **YYYYMMDD_HHMM** (z. B. **20260420_1641**). Power Query promotet jeweils die Kopfzeile und typisiert die Spalten.

3.3.1 iLevels 1 → Tabelle cr9b2_ilevels_1 (I-Stufen, „exploded“)

Kern-ETL der App. Quelle: **..._iLevels 1.csv** (Trennzeichen **;**, 15 Spalten).

- Promote Headers**, Typisierung aller Spalten (**ats**, **fs**, **sab**, **sf** als **datetime**; **number**, **status** als Ganzzahl; Rest Text).
- Split name** am **-** in **name.1** / **name.2**, umbenannt zu **brv** und **iLevel** (z. B. **NA05-26-07-480** → brv **NA05**, iLevel **26-07-480**).
- Spalten entfernen:** **derivative**, danach **brv**, **seriesGroup**, **vehicleType**, **type** (BRV-unabhängiger I-Stufen-Key — gleiche I-Stufe über mehrere BRVs kollabiert).
- Duplikate entfernen** anhand **iLevel** (Distinct).
- Datumstypen** für **ats** / **fs** / **sab** / **sf**, Spalten neu geordnet, **iLevel** als Text.
- Rolling-Window-Filter:** **ats >= heute - 10 Wochen** (**Date.AddWeeks(DateTime.Date(DateTime.LocalNow()), -10)**) — hält nur Softwarestände im relevanten Zeitfenster, dynamisch zur Laufzeit.
- Wochen-Explosion (Unpivot):** **totalWeeks = RoundDown((sab - ats) / 7 Tage)**; es wird eine Liste **{0..totalWeeks}** erzeugt und per **ExpandListColumn** zu **einer Zeile je Woche** expandiert (**WeekIndex**).

8. **OFFSET -Label:** `WeekIndex = 0` → "ATS", `WeekIndex = totalWeeks` → "SAB", sonst "ATS+n".
9. **WEEK (ISO-8601):** aus `ats + WeekIndex · 7 Tagen` wird die ISO-Woche **Thursday-basiert** berechnet (Donnerstag der Woche → Jahr; `jan4` -Montag als Anker) und als "YYYY-WW" (nullgepolstert) ausgegeben.
10. **KW** : dieselbe ISO-Berechnung, nur die reine Wochennummer als Ganzzahl.

Ergebnis: pro I-Stufe so viele Zeilen wie Wochen zwischen ATS und SAB, jeweils mit Offset-Label und ISO-Kalenderwoche. Genau dieses „exploded“ Format liest die Anwendung und normalisiert `ATS / SAB` zu `ATS WEEK / SAB WEEK`.

3.3.2 PlannedComponentApprovals → Tabelle `cr9b2_planned_component_approvals` (Penthouse-Tickets, Jira)

Quelle: `...PlannedComponentApprovals.csv` (Trennzeichen `,`, 28 Spalten — voller Jira-Export inkl. `jiraKey`, `jiraURL`, `dueDate`, `iLevelNames`, `parentJiraIssue`, `parentBranches`, `leadIStep`, `ticketStatus` ...).

1. **Promote Headers**, Typisierung aller 28 Spalten.
2. **Filter Penthouse-Branch:** nur Zeilen mit `parentBranches = ["PTH-06"]` (genau dieses Penthouse).
3. **Rolling-Window-Filter:** `dueDate >= heute - 10 Wochen`.
4. **ISO-Anreicherung:** aus `dueDate` werden per Record `KW`, `Year` und `YearWeek` („YYYY-WW“) berechnet (gleiche Thursday-basierte ISO-Logik) und als Spalten expandiert/typisiert.

3.3.3 Verbundfreigaben → Tabelle `cr9b2_verbundfreigaben` (Verbund-/SWAP-Tickets)

Quelle: `...Verbundfreigaben.csv` (Trennzeichen `,`, 6 Spalten: `iLevelName`, `name`, `startDate`, `endDate`, `collapseId`, `replaced`).

1. **Promote Headers**, Typisierung.
2. **Stichtags-Filter:** `startDate = 2026-04-23` — fest verdrahtetes **Snapshot-Datum** (siehe Risiken §3.3.6).
3. **SWAP-Filter:** nur Zeilen, deren `collapseId` den Text "SWAP" enthält.
4. **BRV-/Baureihen-Filter:** `iLevelName` beginnt mit `G065`, `NA00`, `NA05`, `NA06` oder `RR45`.
5. **Key-Bildung:** `Key = iLevelName-name-collapseId`, anschließend **Distinct** auf `Key`.

3.3.4 Verbundfreigaben_EX (aggregierte Variante, Gruppierung je Collapse)

Wie 3.3.3 bis zur Distinct-Stufe, danach:

1. **Gruppierung nach `collapseId`** mit `Count` (Zeilen je Collapse) und `iLevelNames` (`Text.Combine(List.Distinct(iLevelName), "|")` — alle zugehörigen I-Stufen, pipe-getrennt).
2. **Sortierung** nach `Count` absteigend.

Diese Variante liefert die „ein Collapse → viele I-Stufen“-Sicht, auf der die Verbund-Ticket-Badges der App (Feld `iLevelNames`) basieren.

3.3.5 dimBRV → Tabelle `cr9b2_brv` (BRV-Dimension)

Quelle wie 3.3.3 (Verbund-CSV).

1. Promote/Typisierung, Stichtags-Filter `startDate = 2026-04-23`, SWAP-Filter.
2. **Split** `iLevelName` am `-` in bis zu vier Teile, nur erstes Token (`iLevelName.1` = BRV-Präfix) behalten.
3. **Distinct**, dann Filter auf die Menge `{G065, NA00, NA05, NA06, RR45}`.
4. (Variante „BRV“: zusätzlicher **Rename** `iLevelName.1 → brv`).

3.3.6 Beobachtungen, Annahmen & Risiken des Dataflows

- **Manuelle CSV-Quelle:** Die Pipeline startet an **manuell nach SharePoint hochgeladenen CSVs**, nicht an einer direkten Jira-/CDH-API. Aktualität hängt am manuellen Export/Upload (Dateinamen-Zeitstempel).
- **Hartkodiertes Stichtagsdatum:** Verbund-/BRV-Abfragen filtern fix auf `startDate = 2026-04-23`. Für den laufenden Betrieb sollte dies **parametrisiert** werden (Dataflow-Parameter oder Rolling-Window analog iLevels/PCA).
- **Rolling Window 10 Wochen:** iLevels (`ats`) und PCA (`dueDate`) nutzen ein dynamisches Fenster „ab heute – 10 Wochen“; die App-Wochenfenster (3 Wochen) liegen darin.
- **Penthouse fix auf `PTH-06`** : Nur dieser Penthouse-Branch wird geladen; weitere Penthouses erfordern eine Anpassung des Filters.
- **Baureihen-Whitelist:** `G065/NA00/NA05/NA06/RR45` sind fest verdrahtet; in der App ist BRV zusätzlich auf `NA05` reduziert (§6).
- **ISO-Wochenlogik dupliziert:** Die Thursday-basierte ISO-8601-Berechnung ist in `iLevels` und `PCA` identisch ausgeführt — Kandidat für eine gemeinsame M-Funktion.
- **Nicht im Dataflow enthalten:** Die Tabellen `cr9b2_hvs`, `cr9b2_cdh_ip3`, `cr9b2_ip3_freigaben` und `cr9b2_wbs_type_mapping` werden von „FGC_DATA“ **nicht** befüllt — sie stammen aus separater Pflege bzw. einem weiteren Dataflow (zu klären / zu dokumentieren).

4. Funktionale Anforderungen

Notation: Jede Anforderung trägt eine eindeutige Kennung (F-...) und beschreibt Auslöser, erwartetes Verhalten, zugrunde liegende Regeln und relevante Sonderfälle.

4.0 App-Shell & Navigation

- **F-NAV-1 — Seitenleiste:** Persistente, kategorisierte Navigation mit BMW-Logo und App-Titel „Freigabcockpit / P3 Group · HVS Software“. Kategorie „**Hauptmenü**“ (Cockpit, Freigabe Timeline, Verschränkungen) und „**Daten**“ (Dataverse Admin). Aktiver Link wird hervorgehoben; jeder Eintrag hat ein eigenes Icon.
- **F-NAV-2 — Ein-/Ausklappen:** Die Seitenleiste lässt sich kollabieren; im kollabierten Zustand werden nur Icons mit Tooltip gezeigt, Kategorie-Überschriften weichen Trennern.
- **F-NAV-3 — Routing:** `/cockpit`, `/freigabe-timeline`, `/verschraenkungen`, `/dataverse`. Unbekannte Routen leiten auf `/freigabe-timeline` um. Logo-Klick führt zur Timeline.
- **F-NAV-4 — Ladebildschirm:** Beim Start wird ein animierter LoadingScreen gezeigt, bis die App bereit ist.

4.1 Freigabe Timeline (Gantt-Hauptansicht) — `/freigabe-timeline`

Diese Seite ist die planungsnahe Vollansicht: **alle HVS-Zeilen x ein 3-Wochen-Fenster** in einem Raster.

- **F-TL-1 — Gantt-Raster:** Zeilen = HVS/Speichertyp, Spalten = drei Kalenderwochen (Vorwoche · aktuelle KW · nächste KW). Spaltenkopf zeigt Monatslabel (nur bei Monatswechsel), Kontext-Hinweis („• Vorwoche / ● Aktuelle KW / • Nächste KW“), KW-Nummer und Datumsbereich (Mo–So). Die aktuelle KW-Spalte ist durchgängig hervorgehoben. Zeilen sind zebragestreift; inaktive HVS-Zeilen werden gedämpft dargestellt.
- **F-TL-2 — Wochennavigation:** Buttons „« 3“ / „<“ / „Heute“ / „>“ / „3 »“ verschieben das Fenster um ± 1 bzw. ± 3 Wochen; „Heute“ springt auf Offset 0 und ist dort deaktiviert. Eine zweite, kompakte Wochennavigation sitzt im Label-Kopf des Rasters.
- **F-TL-3 — Zeile „Aktive Softwarestände“:** Pro KW werden alle in dieser Woche aktiven I-Stufen (automatisch aus dem Aktiv-Zeitraum + manuell hinzugefügte) als farbige Chips dargestellt. Jeder Chip zeigt den I-Stufen-Key und das/die Offset-Label(s) der Woche. Die Farbe ist **deterministisch je SE_TERMIN** (stabile Palette von 10 Farben, lexikografisch zugewiesen) — jeder Softwarestand desselben SE_TERMIN teilt dieselbe Farbe, unabhängig vom Reifegrad. Ein Klick auf einen Chip wählt diesen Softwarestand als Bulk-Ziel (Toggle).
- **F-TL-4 — Rang/Gruppe & Lead je Woche:** Auf jedem Chip kann pro KW eine **Gruppen-/Rang-ID** (Select 1..n) und eine **Lead-Markierung** (Checkbox) gesetzt werden. Lead ist innerhalb derselben Rang-Gruppe **exklusiv**: Wird ein Chip auf Lead gesetzt, verlieren die anderen Mitglieder derselben Rang-Gruppe ihren Lead. Rang und Lead sind wochenspezifisch (Key `"weekKey|istufe"`).
- **F-TL-5 — Penthouse-Ticket-Zeile (Di):** Eigene Rasterzeile zeigt je KW die Penthouse-Tickets als Badges, farbcodiert nach abgeleitetem Level (L1=Orange, L2/L3/L4=Grüntöne, RSTB=Gelb, FB=Blau, sonst Grau). Das Badge trägt Ticketname und ggf. Softwarestand. Klick öffnet ein **Detail-Popover** mit: Name, Due-Date, KW, Softwarestand, Parent-Jira-Issue, Parent-Branches, Jira-Deeplink und der Liste zugehöriger I-Stufen (eingefärbt je SE_TERMIN). Eine Zähler-Badge im Zeilenkopf zeigt die Ticketzahl im sichtbaren Fenster (nach ISTUFE-Filter).
- **F-TL-6 — Verbund-Ticket-Zeile (Do):** Analog zu F-TL-5 für Verbund-/Swap-Tickets; Popover zeigt Collapse-ID, Start- und Enddatum, KW und zugehörige I-Stufen.
- **F-TL-7 — CDH-Soll-Badges:** Pro Zelle (HVS x KW) werden aus `cr9b2_cdh_ip3` die geplanten Freigabe-Level („Soll“) als Badges eingeblendet (Key `"startWeek|hvsKey"`), farbcodiert nach Level.
- **F-TL-8 — Geplante Freigaben & Ist-Stand je Zelle:** Pro Zelle werden die aus Penthouse-Tickets dieser Woche abgeleiteten geplanten Freigaben gezeigt — **eine Zeile je BRV-bereinigter I-Stufe** des Tickets. Jede Zeile besteht aus: I-Stufen-Chip (Farbe der SE_TERMIN) inkl. Offset, Soll-Level-Badge (Ticketname) und einem editierbaren **Ist-Stand-Badge**. Klick auf das Ist-Badge öffnet ein Dropdown mit den Werten `- Nicht gesetzt -`, `X`, `RSTB`, `L1`, `L2`, `L3`, `L4`. Bei inaktiver HVS-Zeile ist das Ist-Badge schreibgeschützt.
- **F-TL-9 — Verschieben geplanter Freigaben (Move):** Da die Quelltable Jira-synchron ist, wird sie nie verändert. Stattdessen erfasst die App je verschobener I-Stufe ein **Delta** (`ApprovalMove`): Klick auf den I-Stufen-Chip öffnet einen **Move-Picker** mit auswählbaren Ziel-Kalenderwochen. Der Picker bietet nur Wochen an, die den Softwarestand innerhalb der 25-stufigen Leiter halten (kein Über-/Unterlauf von ATS-8 ... SAB+8); ohne bekannten Softwarestand sind ± 26 Wochen erlaubt. Beim Verschieben:
 - wird der **Softwarestand entsprechend der Wochendifferenz entlang der Leiter** verschoben und im Picker als Vorschau angezeigt,
 - wird der Eintrag in der **Ursprungs-KW unterdrückt** (`movedOutKeys`),
 - erscheint er in der **Ziel-KW** mit Herkunftshinweis „↩ KWnn“ (Tooltip nennt Ursprungswoche und früheren Softwarestand),
 - ist die Verschiebung per **Rückgängig-Button** (↶) umkehrbar; eine erneute Verschiebung derselben (Ticket, I-Stufe) re-zielt den bestehenden Move um, statt zu duplizieren.
- **F-TL-10 — Manuelle Einträge:** Ein „+“-Button je Zelle öffnet einen zweistufigen Picker (1. Softwarestand wählen — aktive zuerst, dann übrige; 2. Offset wählen aus 11 Werten `ATS WEEK ... SAB+2`). Der manuelle Eintrag erscheint als gesondert markierte Zeile in der Zelle und ist per „x“ entfernbar. Duplikate (gleiche Woche/HVS/I-Stufe/Offset) werden verhindert. Manuell hinzugefügte I-Stufen erscheinen auch in der „Aktive Softwarestände“-Zeile.

- **F-TL-11 — HVS Aktiv/Inaktiv-Toggle:** Jede HVS-Zeile hat einen Schalter (initialisiert aus `defaultActive`). Inaktive Zeilen sind schreibgeschützt (kein „+“-Button, Ist-Badges read-only) und visuell gedämpft. Der Zustand fließt in die Bulk-Auswahl ein (nur aktive HVS sind per „Alle aktiven“ wählbar).
- **F-TL-12 — Bulk-Freigabe-Panel (3 Schritte):** Dauerhaft eingeblendetes Panel zum Setzen eines Freigabe-Levels für viele HVS in allen aktiven Wochen:
 - **Schritt 1 — Zielstand:** ISTUFE (SE_TERMIN) → Reifegrad (abhängige Auswahl) → Freigabe-Level (`X`, `RSTB`, `L1-L4`). Aus SE_TERMIN + Reifegrad wird der I-Stufen-Key abgeleitet; ist er gültig, werden die im sichtbaren Fenster **aktiven Wochen** als an-/abwählbare Pills angeboten (mit „Alle/Keine“).
 - **Schritt 2 — Penthouse-Auswahl:** Gruppenselektor; das An-/Abwählen eines Penthouse selektiert bzw. deselektiert automatisch alle (aktiven) HVS dieses Penthouse.
 - **Schritt 3 — HVS-Auswahl:** Einzelauswahl der HVS-Zeilen (Checkliste, „Alle aktiven“/„Keine“).
 - **Vorschau & Bestätigung:** Zeigt ISTUFE, Reifegrad, Softwarestand, ggf. Kaskade, betroffene Wochen, Level und betroffene HVS; erst „Bestätigen & Anwenden“ schreibt.
 - **Anwenden** schreibt das Ist-Level für jede gewählte Kombination HVS × KW. Innerhalb einer Woche werden zusätzlich die **Gruppenmitglieder gleichen Rangs** mit gesetzt (Rang-/Lead-Logik aus F-TL-4).
- **F-TL-13 — Verschränkungs-Kaskade im Bulk:** Ist der gewählte Zielstand **verschränkt**, gilt:
 - Die HVS-Liste des Bulk-Panels wird auf den gebundenen Speicher beschränkt (andere HVS werden ausgeblendet, bereits gewählte, nun unzulässige werden entfernt).
 - Die Freigabe **kaskadiert** zusätzlich auf alle Schwester-Softwarestände **derselben SE_TERMIN mit Reifegrad ≥ dem gewählten** (je Woche nur, soweit sie dort aktiv sind). Ein Hinweis nennt die Bindung und die kaskadierten Stände.
- **F-TL-14 — Filter:** Filterzeile mit Volltextsuche (durchsucht HVS, WBS-Typ, Muster, Penthouse, Key), Dropdowns für Penthouse, WBS-Typ und Muster sowie der ISTUFE-Mehrfachauswahl; „Filter zurücksetzen“ erscheint, sobald ein Filter aktiv ist. Eine Statistikleiste zeigt Anzahl Speichertypen und aktive Softwarestände.
- **F-TL-15 — ISTUFE-Sichtbarkeit & Status:** Die ISTUFE-Auswahl arbeitet **je SE_TERMIN** (ein Toggle schaltet alle Reifegrade des Zyklus). Jeder SE_TERMIN trägt eine Statusmarkierung — standardmäßig voreingestellt `26-07 = Abgebrancht`, `26-11 = Entwicklungszeitschiene` (sonst „Planung“). **Default-Sichtbarkeit:** Beim ersten Laden sind nur `26-07` und `26-11` sichtbar; alle übrigen SE_TERMIN sind ausgeblendet. **Strikte Ticket-Sichtbarkeit:** Ein Penthouse-/Verbund-Ticket wird ausgeblendet, sobald **irgendeine** seiner I-Stufen zu einer ausgeblendeten SE_TERMIN gehört (Tickets ohne I-Stufen bleiben sichtbar).
- **F-TL-16 — WMM-/MIA-Badges je Zeile:** Im Label jeder HVS-Zeile sitzt ein **WMM-Badge** mit status-abhängiger Farbe; Klick öffnet den WMM-Drawer (§4.5). Ein **MIA-Badge** ist als Platzhalter vorhanden (Phase 3).
- **F-TL-17 — Daten aktualisieren:** Ein „Aktualisieren“-Button stößt `POST /_dv_/refresh` an (Dev-Backend; läuft `pac org fetch`, ~2–3 min, ohne Client-Timeout). Während des Laufs wird ein Spinner mit Sekundenzähler gezeigt; bei Erfolg lädt die Seite neu (wertet die aktualisierten JSON-Snapshots aus), bei Fehler wird der Fehlertext im Tooltip angezeigt.
- **F-TL-18 — Summen & Legende:** Eine Summenleiste zeigt Gesamtzahlen (HVS, I-Stufen, Penthouse-Tickets, Verbund-Tickets). Eine Legende erklärt die Level-Farben und listet die aktuell sichtbaren Softwarestände mit Farb-Swatch.
- **F-TL-19 — Popover-/Overlay-Verhalten:** Alle Popover (Tickets, Ist-Dropdown, Move-Picker, Add-Picker, ISTUFE-Menü) berechnen ihre **horizontale Platzierung viewport-abhängig** (links/zentriert/rechts) und schließen bei Klick außerhalb sowie mit Escape.

4.2 Cockpit (aktionsorientierte Kartenansicht) — `/cockpit`

Dieselben Daten wie die Timeline, aber **eine Karte je Speicher** und auf schnelles Setzen einzelner Freigaben optimiert. Der Ist-Stand dieser Seite ist (Stand heute) vom Timeline-Stand isoliert (siehe §7, R-2).

- **F-CP-1 — Kopf & Navigation:** Titel „Cockpit“ mit Untertitel, Wochennavigation („< / Heute / >“) und Link „Klassische Timeline ↗“.
- **F-CP-2 — Filterzeile:** Volltextsuche plus ISTUFE-Mehrfachauswahl (als ausklappbares `<details>`-Menü mit Farb-Swatch, Status-Pill und Sichtbarkeits-Checkbox je SE_TERMIN). Default-Sichtbarkeit wie in der Timeline.
- **F-CP-3 — Attention-Kacheln:** Fünf Kennzahl-Kacheln oberhalb der Karten: **WMM-Deltas zu prüfen**, **Sachnummern fehlen** (warnt farblich, wenn > 0), **PTH-Tickets im Zeitfenster**, **Verbund-Tickets im Zeitfenster**, **Speichermuster sichtbar**. Die WMM-Zahlen werden aus dem WMM-Status aggregiert, die Ticketzahlen aus dem sichtbaren 3-Wochen-Fenster.
- **F-CP-4 — ISTUFE-Band:** Ein farbiges Ribbon listet die im aktuellen Fenster sichtbaren Streams als Pills (mit Status-Tooltip).
- **F-CP-5 — Speicher-Karten:** Je sichtbarem HVS eine Karte mit Kopf (HVS · WBS · Muster · ggf. „PTH x“), **WMM-Badge** (status-farbig, öffnet Drawer) und **MIA-Badge** (gedämpfter Platzhalter, „Phase 3“).
- **F-CP-6 — KW-Zellen je Karte:** Drei KW-Zellen mit KW-Nummer und Datumsbereich. Je Zelle wird der **aktive Softwarestand** (höchster sichtbarer Reifegrad) als farbiges Band mit Offset gezeigt. Auf die Zelle/Woche passende **PTH-/Verbund-Tickets** erscheinen als klickbare Badges (öffnen Ticket-Popover). Gibt es keine aktive I-Stufe, zeigt die Zelle „— Keine aktive ISTUFE“.
- **F-CP-7 — Inline-Freigabe je Zelle:** Statt Modal ein **Inline-Level-Picker** (Werte `ø`, `X`, `RSTB`, `L1-L4`) direkt an der Zelle; das gesetzte Level färbt den Button.

- **F-CP-8 — Verschränkungssperre:** Ist der Softwarestand der Zelle an einen **anderen** Speicher verschränkt, wird die Zelle als „ {Speicher}“ gesperrt (keine Eingabe möglich), mit erläuterndem Tooltip.
- **F-CP-9 — Bulk-Freigabe-Sheet:** Ein von rechts einschwebendes Sheet (analog Timeline-Bulk) mit Schritten Zielstand → Wochen → HVS, inklusive **Verschränkungs-Kaskade** und Vorschau/„Anwenden“. Die Wochen sind auf die für den Zielstand aktiven Wochen des Fensters begrenzt.
- **F-CP-10 — Sticky Action-Bar:** Am unteren Rand fixiert: Link „Verschränkungen“ und Button „+ Bulk-Freigabe“.
- **F-CP-11 — WMM-Drawer & Ticket-Popover:** Kompakter, in die Seite integrierter WMM-Drawer (nutzt denselben WMM-Service) und dasselbe Ticket-Popover wie die Timeline.

4.3 Verschränkungen — [/verschraenkungen](#)

- **F-VS-1 — Builder (3 Schritte):** SE_TERMIN → Reifegrad (abhängig) → Speicher (HVS-Kurzcode, abhängig). Sobald eine gültige Kombination gewählt ist, zeigt eine **Live-Vorschau** [ISTUFE](#) → [Speicher](#).
- **F-VS-2 — Speichern mit Duplikatprüfung:** „Verschränkung speichern“ legt die Bindung an. Existiert die Kombination (I-Stufe + Speicher) bereits, erscheint eine Fehlermeldung; bei Erfolg eine Bestätigung. Die I-Stufen-Auswahl bleibt erhalten, damit weitere Bindungen für denselben Softwarestand schnell folgen können.
- **F-VS-3 — Liste aktiver Verschränkungen:** Gruppiert je Softwarestand, mit Anzahl Bindungen, Speicher-Code, Erstell-Zeitstempel (de-DE) und „X“-Löschen je Eintrag. Leerzustand mit Hinweistext.
- **F-VS-4 — Persistenz & Sync:** lokaler Browser-Speicher, tab-übergreifend synchronisiert. Auswirkung der Bindungen siehe F-TL-13, F-CP-8, F-CP-9 (Kaskade/Sperre).

4.4 Dataverse Admin — [/dataverse](#)

- **F-DV-1 — Tabellenauswahl:** Dropdown über die acht [cr9b2_*](#)-Tabellen (Default [cr9b2_wbs_type_mapping](#)).
- **F-DV-2 — Liste:** Lädt bis zu 200 Datensätze ([\\$top=200](#)) über [/_dv/{entity}](#). Spalten werden dynamisch aus dem ersten Datensatz abgeleitet; OData-Annotationen und **Systemfelder** (createdon, modifiedon, owner..., statecode etc.) sowie der Primärschlüssel werden für die Bearbeitung ausgeblendet. Zähler zeigt Zeilen- und Spaltenzahl.
- **F-DV-3 — CRUD:** „+ New“ legt einen Datensatz an (Formular aus editierbaren Feldern; leere Felder werden beim Anlegen weggelassen). „Edit“ bearbeitet (PATCH auf den Primärschlüssel). „Delete“ löscht nach Bestätigungsdialog. Alle Operationen laufen über den Dev-Proxy.
- **F-DV-4 — Fehler- & Ladezustände:** Fehlermeldungen aus dem Backend werden angezeigt; „Loading...“, Leerzustände und ein deaktiviertes „+ New“ bei leerer Tabelle (keine Schema-Inferenz möglich).

4.5 WMM-Verwaltung (derzeit simuliert)

Bis eine echte WMM-Anbindung existiert, bildet das Modul die benötigte Oberfläche nach. Die Schnittstelle des WMM-Service bleibt bei späterer Backend-Anbindung unverändert.

- **F-WMM-1 — Datensatz je Speichermuster:** Jede HVS-Zeile besitzt einen WMM-Datensatz mit ZSMB-ID ([ZSMB-{pnummer}-{muster}](#)), Deeplink (<https://wmm.bmw.intra/zsmb/...>), Status, Sachnummern-Liste und Zeitstempel der letzten Prüfung. Datensätze werden bei Erstzugriff automatisch **geseedet** (3–5 deterministische Sachnummern je HVS).
- **F-WMM-2 — Statusmodell:** [OK](#), [ZSMB fehlt](#), [Sachnummern fehlen](#), [Delta zu prüfen](#), [Deaktiviert](#). Der Status leitet sich konsistent aus dem Datenzustand ab (z. B. leere Liste → „Sachnummern fehlen“).
- **F-WMM-3 — WMM Check (Delta):** Simuliert den Abruf aus **BATKAS/IHP** und berechnet ein Delta gegen den gespeicherten Stand mit drei Klassen: **hinzugefügt / entfernt / geändert** (Label- oder Typänderung). Liegt ein Delta vor, wechselt der Status auf „Delta zu prüfen“ und das Delta wird am Datensatz geparkt.
- **F-WMM-4 — Delta übernehmen/verwerfen:** Ein Banner im Drawer zeigt das Delta nach Klassen gegliedert. „Übernehmen“ mergt es in die Sachnummern-Liste (Entfernte raus, Geänderte ersetzt, Neue ergänzt), „Verwerfen“ lässt die Liste unverändert. Danach wird der Status neu bestimmt.
- **F-WMM-5 — Sachnummern manuell pflegen:** Hinzufügen (Nummer, Typ, Beschreibung; Duplikate werden verhindert) und Entfernen einzelner Sachnummern direkt im Drawer.
- **F-WMM-6 — Deaktivieren/Reaktivieren:** Ein Speichermuster kann im WMM-Sinn deaktiviert und wieder reaktiviert werden.
- **F-WMM-7 — Persistenz & Sync:** lokaler Browser-Speicher, automatische Initialbefüllung für neue HVS-Zeilen, tab-übergreifende Synchronisation.

5. Nicht-funktionale Anforderungen

- **NF-1 Plattform:** Microsoft Power Apps **Code App**; Dataverse als System of Record; `@microsoft/power-apps` Runtime + `@microsoft/power-apps-vite` Plugin; Deployment via `pac code push`.
 - **NF-2 Technologie:** React 19, TypeScript (strict, projektweise tsconfig), Vite 7, react-router-dom 7; ESLint mit react-hooks/react-refresh-Regeln. **Keine** automatisierte Test-Suite konfiguriert.
 - **NF-3 Sprache & Markenauftritt:** Durchgängig **Deutsch**; BMW-Logo und Farbwelt; konsistente Level- und SE_TERMIN-Farbsysteme; responsive Popover-Platzierung; barrierearme Dialoge (`role="dialog"`), Escape-Close, aria-Labels).
 - **NF-4 Performance:** Nur 3-Wochen-Fenster gleichzeitig gerendert; durchgehende Memoisierung abgeleiteter Strukturen (`useMemo` / `useCallback`); committete JSON-Snapshots ermöglichen sofortige Erstanzeige ohne Online-Roundtrip.
 - **NF-5 Datenaktualisierung:** Snapshot-Refresh über `pac org fetch` (~2–3 min) ohne Client-Timeout, mit Fortschritts- und Fehleranzeige.
 - **NF-6 Zeit-/Kalenderkorrektheit:** ISO-Wochen, Montag-basiert; korrekte Behandlung von Jahresgrenzen über ISO-Wochenjahr; einheitliches Schlüsselformat `"YYYY-WW"`.
 - **NF-7 Robustheit:** Fehlertolerante Persistenzschicht (defensives Lesen des lokalen Speichers, robustes Parsen der Verschiebungsliste, automatische Initialbefüllung fehlender Datensätze).
 - **NF-8 Wartbarkeit:** Klare Trennung von Daten, Seiten/Views, wiederverwendbaren Komponenten und der Dataverse-Anbindung; simulierte Module kapseln ihre Backend-Annahmen hinter stabilen Schnittstellen, sodass ein späterer Austausch ohne Änderung der Oberfläche möglich ist.
-

6. Abgrenzung (nicht im aktuellen Stand enthalten)

- Keine rollenbasierte Zugriffssteuerung / Authentifizierungslogik in der App selbst.
 - Keine Schreibrückführung in die Jira-synchronen Tabellen (`planned_component_approvals` , `verbundfreigaben`).
 - Keine echte WMM-, MIA-, DEEP-, Stücklisten- oder Freigabemplate-Anbindung (teils simuliert, teils offen).
 - Kein gemeinsamer Ist-Stand-Store zwischen Cockpit und Timeline (jede Seite hält ihren eigenen Stand).
 - Keine Mehr-BRV-Unterstützung (BRV ist auf `NA05` fixiert).
-

7. Offene Punkte & Roadmap

- **R-1** Persistenz von WMM, Verschränkungen und Freigabe-Verschiebungen vom lokalen Browser-Speicher auf Dataverse-Tabellen migrieren (z. B. `cr9b2_verschraenkungen`, `cr9b2_pth_approval_move`); die Schnittstellen der App bleiben unverändert.
 - **R-2** Ist-Stand zwischen Timeline und Cockpit auf einen gemeinsamen Store/Dataverse heben, damit beide Ansichten dieselbe Wahrheit mutieren.
 - **R-3 MIA-Integration** (technische Bewertung, 5 Stufen) — UI-Platzhalter vorhanden, Funktion offen.
 - **R-4** SE_TERMIN-Status (Abgebrannt/Entwicklung/Planung) aus einem Dataverse-Lookup statt Hardcode beziehen.
 - **R-5** Mehr-BRV-Fähigkeit (dynamische BRV-Auflösung über `cr9b2_brv` statt fixem `NA05`).
 - **R-6** Echte WMM-Anbindung (BATKAS/IHP) statt Simulation.
 - **R-7** Weitere Integrationen laut Gesamtscope: Jira-Automatisierung (Ticketerzeugung/Feldpflege), Freigabetemplate (Word-Generierung + SharePoint + tägliches Update), DEEP-Kurzfristplanung, Stücklisten/CDH, Reporting-/KPI-Dashboards.
 - **R-8 Dataflow „FGC_DATA“ härten:** hartkodiertes `startDate = 2026-04-23` parametrisieren (Rolling Window), CSV-Quelle durch direkte Jira-/CDH-Anbindung ersetzen, ISO-Wochenlogik als gemeinsame M-Funktion faktorisieren, und die nicht abgedeckten Tabellen (`cr9b2_hvs`, `cr9b2_cdh_ip3`, `cr9b2_ip3_freigaben`, `cr9b2_wbs_type_mapping`) in einen dokumentierten Dataflow überführen (siehe §3.3.6).
-

8. Rollen (vorgesehen)

Rolle	Aufgaben / Verantwortung
Release Manager (ES-6)	Steuert die I-Stufen-Freigaben, validiert Termine, erzeugt Freigabedokumente und triggert die Verbund-Weitergaben an Folgesysteme.
CE / Fachingenieur (Komponenten)	Pflegt manuelle Zuordnungen (XCP-/XETK-Nummern, IP3-/SNR-Mapping) und bestätigt die Stücklisten-Relevanz der Sachnummern.
WMM-Steward	Prüft und übernimmt SNR-Updates, verwaltet Zusammenbauten und deren Zuordnungen im WMM.
MIA-Koordinator	Prüft und steuert MIA-Anlagen, Zuordnungen und Level-Freigaben.
PM / Ops	Monitoring, Reporting, Eskalationen.
Admin / Platform	Benutzer- und Rollenverwaltung, Verbindungen/Connectoren, Policies.

(Rollenbasierte Sichtbarkeits- und Schreibrechte sind in der aktuellen Version noch nicht umgesetzt — siehe §6.)

9. Kundenanforderungen & Umsetzungsstand

Dieser Abschnitt nimmt den Anforderungskatalog des Kunden auf und bewertet jede Position anhand des aktuellen Funktions- und Datenstands. Statusschlüssel:

- **Umgesetzt** — vollständig vorhanden und nutzbar.
- **Teilweise** — Grundfunktion bzw. UI vorhanden; eine produktive Quelle/Anbindung fehlt noch.
- **Offen** — fachlich vorgesehen, aber noch nicht umgesetzt.
- **Blockiert** — abhängig von einer externen Schnittstelle (Jira / MIA / BATKAS-IHP), die derzeit nicht verfügbar ist.

9.0 Allgemein

Nr	Anforderung	Status	Bewertung & Begründung
—	Filterung auf 26-07 (Abgebrannt) und 26-11 (Entwicklungszeitschiene)	Umgesetzt	Default-Sichtbarkeit des ISTUFE-Filters ist exakt auf 26-07 und 26-11 gesetzt, beide mit Status-Badge (Abgebrannt / Entwicklungszeitschiene). Weitere Zeitschienen sind ausgeblendet und jederzeit zuschaltbar.

9.1 WMM

Nr	Anforderung	Status	Bewertung & Begründung
1	Jeder im WMM gepflegte Speicher im FC sichtbar; Direktlink zum Zusammenbau; Deaktivierung möglich	Teilweise	ZSMB-Deeplink und Deaktivierung je Speicher sind umgesetzt (WMM-Drawer). Die Sichtbarkeit beruht derzeit auf den HVS-Stammdaten bzw. simulierten WMM-Daten, nicht auf einer Live-WMM-Quelle.
2	FC fügt neue WMM-Zusammenbauten automatisch ein	Offen	Eine automatische Initialbefüllung je HVS-Zeile existiert; das echte Nachziehen NEUER Zusammenbauten aus dem WMM erfordert die WMM-Schnittstelle (nicht angebunden).
3	Speicherzeilen im FC anlegbar; Hinweis auf korrekte ZSMB-Zuordnung	Teilweise	Anlegen ist über die Dataverse-Verwaltung möglich; der Status „ZSMB fehlt“ wird angezeigt. Ein geführter Zuordnungs-Workflow direkt im Cockpit ist noch offen.
4	Kein ZSMB vorhanden → Weiterleitung zur WMM-Landingpage oder leeren ZSMB erzeugen und Link hinterlegen	Offen	Erfordert die WMM-Schnittstelle; aktuell nur manueller ZSMB-Link und manuelle Pflege.
5	FC informiert, wenn ZB-HVS-Sachnummern im ZSMB fehlen (Eintrag durch CE)	Umgesetzt (UI)	Status „Sachnummern fehlen“ je Speicher plus aggregierte Kennzahl in den Attention-Kacheln. Die Eintragung erfolgt extern im WMM durch CE.
6	Aus ZB-HVS in BATKAS/IHP die Stückliste auslesen, relevante Sachnummern (Type/Kackel) vorschlagen, bei Mehrfachtreffern User-Auswahl, anschließend ins WMM befüllen	Blockiert	Die Vorschlags-, Auswahl- und Delta-Mechanik ist im UI als Konzept abgebildet (simuliert); die reale Stücklisten-Anbindung an BATKAS/IHP fehlt.
7	„WMM Check“-Schaltfläche je Speicher: Sachnummern neu auslesen, Deltabewertung, Delta an User	Teilweise	„WMM Check“ inkl. Deltabewertung (hinzugefügt/entfernt/geändert) sowie Übernehmen/Verwerfen ist vollständig umgesetzt — derzeit gegen eine simulierte Quelle. Die Logik steht; die produktive Datenquelle ist offen.

9.2 IP3

Nr	Anforderung	Status	Bewertung & Begründung
8	FC durchsucht IP3 regelmäßig nach neuen P-Nummern im Cluster Gen6 HVS	Offen	Kein automatisches IP3-Polling. IP3-Daten liegen als Tabelle vor, werden aber derzeit nicht über den Dataflow „FGC_DATA“ befüllt (siehe §3.3.6).
9	Neues Speicherprojekt → neuer Block im FC; Deaktivierung möglich	Teilweise	Deaktivierung über den Aktiv/Inaktiv-Schalter je HVS-Zeile ist umgesetzt; das automatische Erzeugen neuer Blöcke aus IP3 erfordert das IP3-Polling.
10	Geplante Speichermusterphasen mit Terminierung (L2, L3) automatisch hinzufügen	Teilweise	L2/L3-Solltermine werden je Zelle als CDH-Soll-Badges angezeigt (Quelle <code>cr9b2_cdh_ip3</code>); das automatische Anlegen aus einem IP3-Lauf ist offen.
11	Jede Speichermusterphase benötigt einen WMM-Zusammenbau (Vorgehen aus Punkt 4)	Offen	Abhängig von der WMM-Anbindung (siehe Punkt 4).

Nr	Anforderung	Status	Bewertung & Begründung
12	L2/L3-Termine aus IP3 sind Grobplanung; Verschieben auf andere KWs muss möglich sein	Umgesetzt	Verschieben über den Move-Picker (Softwarestand-Leiter, Herkunftsmarkierung „↔ KWnn“, Rückgängig).

9.3 DEEP

Nr	Anforderung	Status	Bewertung & Begründung
13	Verbundfreigabeplanung aus DEEP dynamisch sichtbar; Due-Date-Änderung führt zu Verschiebung	Teilweise	Verbund-Tickets sind sichtbar (Verbund-Zeile, read-only Datenstand). Änderungen am Ticket-Due-Date führen derzeit NICHT automatisch zu einer Verschiebung — diese werden manuell erfasst. Der automatische Abgleich ist offen.
14	Zugehörigkeit zur richtigen Zeitschiene optisch ersichtlich und logisch	Umgesetzt	Deterministische Farbe je SE_TERMIN plus Status-Badges; Zeitschienen sind klar getrennt erkennbar.
15	PTH-Tickets (Bernd) sichtbar; Ziel: PTH-Ticket-Erzeugung aus dem FC	Teilweise	PTH-Tickets sind sichtbar (Penthouse-Zeile, Popover, Jira-Link). Die Erzeugung aus dem FC heraus erfordert die Jira-Anbindung (offen).
16	Batch-Change: in KW X allen Speichern mit PTH Y eine L2 in I-Stufe Z geben — FC befüllt sich selbst	Umgesetzt	Die Bulk-Freigabe leistet genau dies (Zielstand + Wochen + HVS-/Penthouse-Auswahl), inklusive Verschränkungs-Kaskade.
17	Einzelne Termine nach dem Batch-Change anpassbar (z. B. L2 aus einem bestimmten Speicher entfernen)	Umgesetzt	Ist-Stand je Zelle editierbar; manuelle Einträge und Verschiebungen sind einzeln entfernbar.
18	FC zieht aus PTH-Tickets, welche Muster diese Woche freigegeben werden, und befüllt sich selbst	Offen	Tickets werden gelesen und KW-genau zugeordnet; die vollautomatische Selbstbefüllung erfordert die Jira-Anbindung.

9.4 Jira

Nr	Anforderung	Status	Bewertung & Begründung
19	Nach Punkt 16: je PTH-Muster ein Ticket im FUSI-Jira erzeugen (I-Stufe + Speicher mit WMM-Links verknüpft)	Blockiert	Nicht umgesetzt; Zielstand bei funktionierender Jira-Schnittstelle.

9.5 MIA

Nr	Anforderung	Status	Bewertung & Begründung
20	Nach Punkt 16: MIA-Vorgang als Entwurf erzeugen (Titel = I-Stufe, alle Speichermuster verknüpft; Befüllung durch User)	Blockiert	Nicht umgesetzt; scheitert aktuell an der MicrosoftxMIA-Anbindung (Kundenhinweis). Ein MIA-Badge ist als Platzhalter vorhanden.
21	Je Speichermuster einen HW-MIA-Link sicherstellen; bei Bestandsaufnahme gibt der User den Link, FC legt ihn ab	Blockiert	Nicht umgesetzt; die MIA-Anbindung fehlt.
22	Neues Speichermuster → neuer MIA-Vorgang (Bezeichnung automatisch, Terminierung aus FC, Komponenten vererbt oder leer)	Blockiert	Nicht umgesetzt; die MIA-Anbindung fehlt.
23	MIA-Bewertungsstatus über das FC einsehbar	Offen	Noch nicht umgesetzt; laut Kundenhinweis „möglich ab Juni“ (abhängig von der Anbindung).

Zusammenfassung: Der gesamte interne Planungs- und Pflege-Kern (Filterung, Zeitschienen-Visualisierung, Batch-Change/Bulk-Freigabe, Einzelanpassung, Terminverschiebung, WMM-Check-Logik) ist umgesetzt. Offen oder blockiert sind durchgängig genau die Punkte, die eine **externe Live-Anbindung** voraussetzen — WMM/BATKAS/IHP (Punkte 1–4, 6, 11), IP3-Polling (8–10), Jira (15, 18, 19) und MIA (20–23). Diese sind nicht durch die Anwendung limitiert, sondern durch die noch fehlenden Schnittstellen.

10. Datenmodell (ER-Diagramm)

Die Dataverse-Tabellen sind physisch **flach** modelliert (keine erzwungenen Fremdschlüssel-Constraints); die Beziehungen sind **logische Joins**, die in der Datenaufbereitung bzw. in der Anwendung ausgeführt werden. Mehrere I-Stufen-Bezüge liegen als **Mehrwert-Felder** vor (pipe-separierte `iLevelNames`) und bilden damit faktische M:N-Beziehungen. Die app-lokalen Entitäten (WMM, Verschränkung, Verschiebung, Ist-Stand) sind für eine spätere Überführung nach Dataverse vorgesehen (§7, R-1).

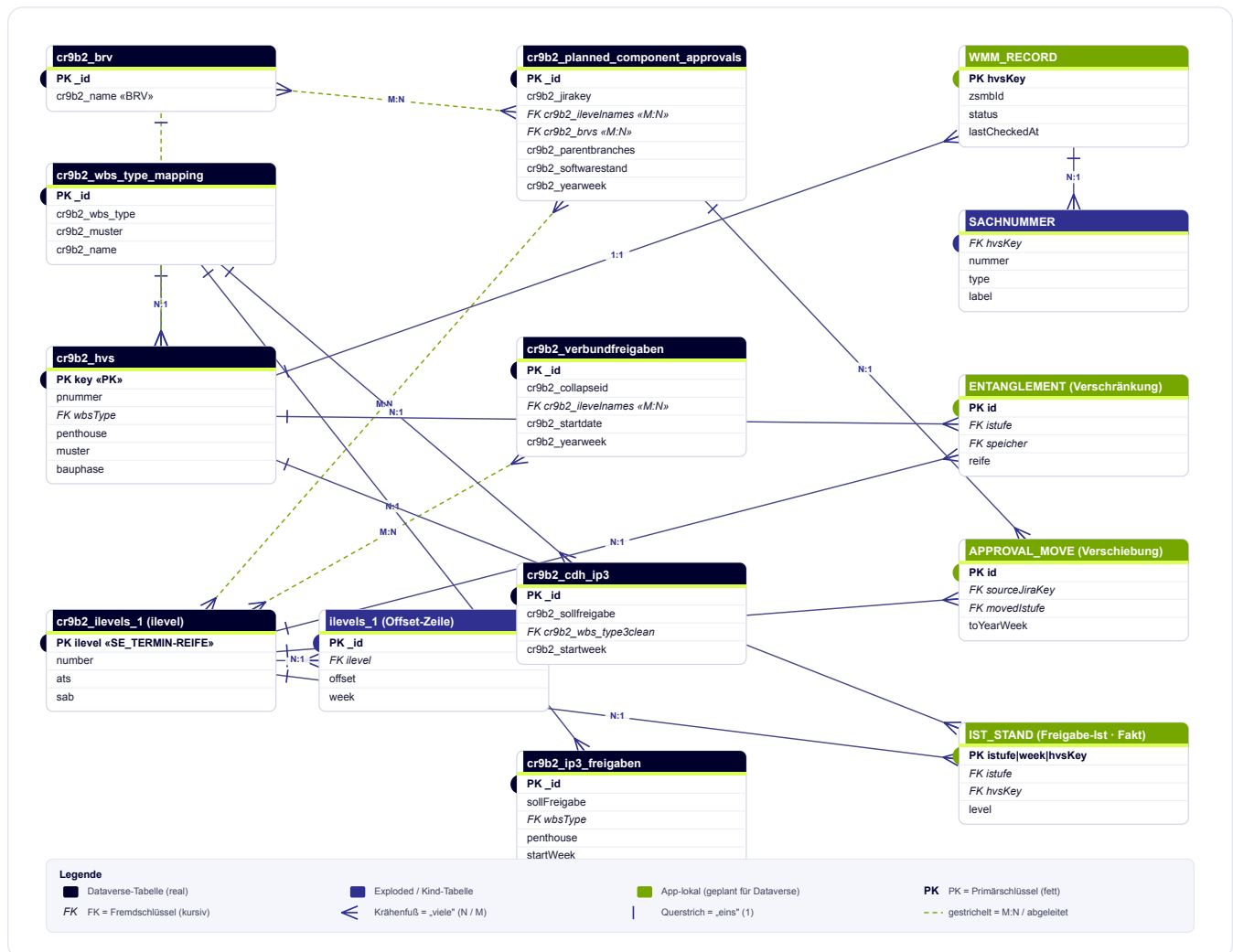


Abb. 1 — Datenmodell Freigabcockpit Next (ER-Diagramm, Krähfuß-Notation)

10.1 Entitäten (Auswahl der Schlüsselattribute)

Entität	Typ	Schlüssel / wichtige Felder
<code>cr9b2_brv</code>	Stammdaten	PK <code>_id</code> ; <code>cr9b2_name</code> (BRV, z. B. NA05)
<code>cr9b2_wbs_type_mapping</code>	Stammdaten	PK <code>_id</code> ; <code>cr9b2_wbs_type</code> , <code>cr9b2_muster</code> , <code>cr9b2_name</code>
<code>cr9b2_hvs</code>	Stammdaten	PK <code>key</code> (= <code>pnummer</code> - <code>wbsType</code>); <code>pnummer</code> , <code>wbsType</code> (FK), <code>penthouse</code> , <code>muster</code> , <code>bauphase</code>
<code>cr9b2_ilevels_1</code>	Stammdaten (exploded)	logische Einheit <code>ilevel</code> (SE_TERMIN-REIFE); je Zeile <code>offset</code> , <code>week</code> , <code>ats</code> , <code>sab</code>
<code>cr9b2_cdh_ip3</code>	Termine (Soll)	PK <code>_id</code> ; <code>cr9b2_sollfreigabe</code> , <code>cr9b2_wbs_type3clean</code> (FK), <code>cr9b2_startweek</code>
<code>cr9b2_ip3_freigaben</code>	Termine (Soll)	PK <code>_id</code> ; <code>sollfreigabe</code> , <code>wbsType</code> (FK), <code>penthouse</code> , <code>startWeek</code> , <code>sop</code>

Entität	Typ	Schlüssel / wichtige Felder		
cr9b2_planned_component_approvals	Tickets (Jira/PTH)	PK _id ; cr9b2_jirakey , cr9b2_ilevelnames (M:N), cr9b2_brvs , cr9b2_parentbranches , cr9b2_softwarestand , cr9b2_yearweek		
cr9b2_verbundfreigaben	Tickets (DEEP/SWAP)	PK _id ; cr9b2_collapseid , cr9b2_ilevelnames (M:N), cr9b2_startdate , cr9b2_yearweek		
WMM_RECORD	App-lokal	PK hvsKey (FK→ hvs.key); zsembId , status , lastCheckedAt		
SACHNUMMER	App-lokal (Kind)	FK hvsKey (→ WMM_RECORD); nummer , type , label		
ENTANGLEMENT (Verschränkung)	App-lokal	PK id ; istufe (FK→ ilevel), speicher (FK→ hvs), reife		
APPROVAL_MOVE (Verschiebung)	App-lokal	PK id ; sourceJiraKey (FK→PTH), movedIstufe (FK→ ilevel), toYearWeek , active		
IST_STAND (Freigabe-Ist, Fakt)	App-lokal	PK istufe \	week \	hvsKey ; FK→ ilevel , FK→ hvs , level

10.2 Beziehungen

#	Von (FK/Mehrwert-Feld)	→ Nach	Kardinalität	Art
R1	hvs.wbsType	wbs_type_mapping.cr9b2_wbs_type	N:1	logischer Join
R2	hvs.briv (= NA05)	briv.cr9b2_name	N:1	abgeleitet (fix NA05)
R3	ilevels_1 (Zeile)	ilevel (logische Einheit)	N:1	Explosion je Offset/Woche
R4	planned_component_approvals.cr9b2_ilevelnames	ilevel	M:N	Mehrwert-Feld, BRV-Strip
R5	planned_component_approvals.cr9b2_brvs	briv	M:N	Mehrwert-Feld
R6	verbundfreigaben.cr9b2_ilevelnames	ilevel	M:N	Mehrwert-Feld; collapseid gruppiert
R7	cdh_ip3.cr9b2_wbs_type3clean	wbs_type_mapping	N:1	logischer Join (Soll je WBS)
R8	ip3_freigaben.wbsType	wbs_type_mapping	N:1	logischer Join
R9	WMM_RECORD.hvsKey	hvs.key	1:1	App-Join
R10	SACHNUMMER	WMM_RECORD	N:1	Eltern-Kind (lokal)
R11a	ENTANGLEMENT.istufe	ilevel	N:1	App-Join
R11b	ENTANGLEMENT.speicher	hvs	N:1	App-Join (Kurzcode)
R12a	APPROVAL_MOVE.sourceJiraKey	planned_component_approvals.cr9b2_jirakey	N:1	App-Join
R12b	APPROVAL_MOVE.movedIstufe	ilevel	N:1	App-Join
R13a	IST_STAND.hvsKey	hvs.key	N:1	Fakt → Dimension
R13b	IST_STAND.istufe	ilevel	N:1	Fakt → Dimension

10.3 Mermaid-Quelltext (zur Weiterverwendung)

```
erDiagram
    BRV ||--o{ HVS : "brv (fix NA05)"
    WBS_TYPE_MAPPING ||--o{ HVS : "wbsType"
    WBS_TYPE_MAPPING ||--o{ CDH_IP3 : "wbs_type3clean"
    WBS_TYPE_MAPPING ||--o{ IP3_FREIGABEN : "wbsType"
    ILEVEL ||--o{ ILEVEL_WEEK : "offset/week (exploded)"
    ILEVEL }o--o{ PLANNED_COMPONENT_APPROVALS : "iLevelNames (M:N)"
    BRV }o--o{ PLANNED_COMPONENT_APPROVALS : "brvs (M:N)"
    ILEVEL }o--o{ VERBUNDFREIGABEN : "iLevelNames (M:N)"
    HVS ||--|| WMM_RECORD : "hvsKey"
    WMM_RECORD ||--o{ SACHNUMMER : "sachnummern"
    ILEVEL ||--o{ ENTANGLEMENT : "istufe"
    HVS ||--o{ ENTANGLEMENT : "speicher"
    PLANNED_COMPONENT_APPROVALS ||--o{ APPROVAL_MOVE : "sourceJiraKey"
    ILEVEL ||--o{ APPROVAL_MOVE : "movedIstufe"
    HVS ||--o{ IST_STAND : "hvsKey"
    ILEVEL ||--o{ IST_STAND : "istufe"
```

11. Zielbild & Systemlandschaft (Gesamtscope)

Dieser Abschnitt dokumentiert das **vollständige Zielbild** von FreigabcockpitNEXT laut Anforderungsdokument der Abteilung ES-6: alle Quell- und Zielsysteme, die fünf Funktionssäulen, die Schlüsselkonzepte und das Epic-/Feature-Backlog. Er beschreibt den **Soll-Umfang** — der aktuelle Umsetzungsstand ist in §4 (Funktionen) und §9 (Anforderungsbewertung) ausgewiesen; je Funktionssäule ist der Bezug zum heutigen Stand vermerkt.

11.1 Prozess- & Systemkontext

FreigabcockpitNEXT ist die **zentrale Drehscheibe** zwischen den Planungs-/Wissens-Quellsystemen und den nachgelagerten Freigabe-Zielsystemen. Der End-to-End-Use-Case verläuft in vier Phasen: (1) **Datenaufnahme & Abgleich** aus IP3/CDH, Stücklisten/CDH und DEEP inkl. Deduplizierung und Differenzlogik; (2) **WMM-Pflege** — Prüfen/Anlegen des Zusammenbaus (ID = ZB_HVS) und Zuordnen der Sachnummern; (3) **Planung & Freigabe** im User-Interface (Übersichtsgrid, Live-Filter, Verbundbildung, Batch-Change, Ist-Freigabe); (4) **Output-Erzeugung** in den Zielsystemen (Jira-Tickets samt Epic, MIA-Anlagen, Freigabemplate nach SharePoint, FUSI Confluence). Die Rollen (Release Manager, CE, WMM-Steward, MIA-Koordinator) wirken vor allem in Phase 3.

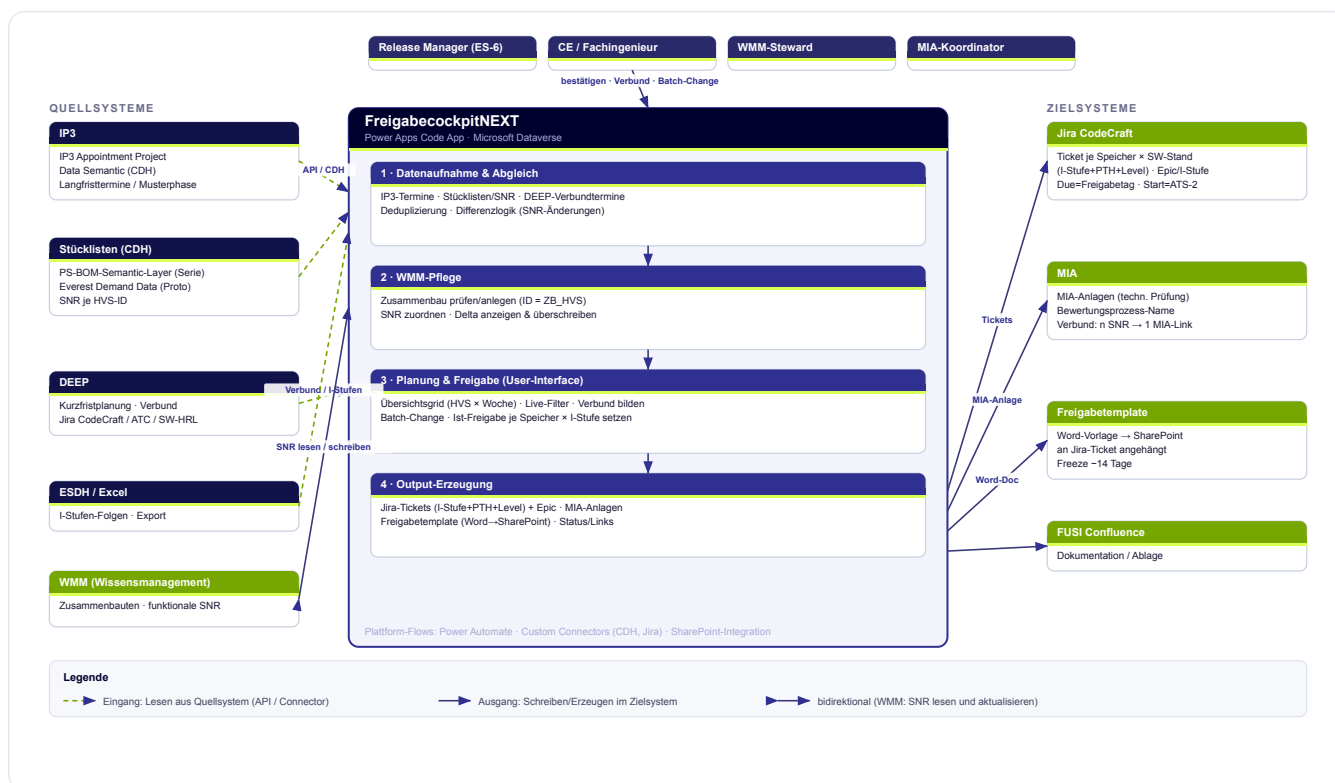


Abb. 2 — Prozess- und Systemkontext: Quellsysteme → FreigabcockpitNEXT → Zielsysteme (End-to-End-Use-Case)

11.2 Funktionssäulen

11.2.1 App — Übersicht & Freigabebestätigung

Die App ersetzt das Excel „Freigabcockpit“ der ES-6. Statt manueller Befüllung greift sie Informationen über automatisierte Schnittstellen aus den Quellsystemen ab, lässt im UI die Freigabetermine für Hochvoltspeicher bestätigen und befüllt anschließend auf Basis der gesammelten Daten und der Nutzereingaben die Folgesysteme. *Bezug zum aktuellen Stand:* Übersichtsgrid, Filter, Batch-Change und Ist-Bestätigung sind umgesetzt (§4.1, §4.2); die automatisierten Schnittstellen sind aktuell durch den Dataflow „FGC_DATA“ (Snapshot/CSV) bzw. Simulationen abgebildet (§3.3, §9).

11.2.2 WMM — Wissensmanagement

Im WMM sollen perspektivisch nur **funktional relevante Sachnummern** dem Zusammenbau zugeordnet werden. IP3-Nummern und Sachnummern (SNR) sind nur manuell zuordnungsbar — dafür schafft die App ein Eingabefeld. Speicherprojekte sind im WMM vorangelegt; ein Flow prüft, ob bereits ein Zusammenbau enthalten ist. **Nach keinem Zusammenbau zugeordnete SNR** werden in der App herausgestellt; **Updates** der zugeordneten Sachnummern werden dem Nutzer angezeigt und auf Knopfdruck überschrieben. Ist einem vorangelegten Speicherprojekt noch kein Zusammenbau zugeordnet, wird dieser per Workflow angelegt. Eindeutige ID ist stets die **ZB_HVS-Nummer**. *Bezug zum aktuellen Stand:* WMM-Drawer mit Status, Delta-Bewertung und „Überschreiben“ ist umgesetzt (gegen simulierte Quelle, §4.5, §9.1); das Anlegen von Zusammenbauten (ZB_HVS) und die Live-WMM-Anbindung sind offen.

11.2.3 Jira CodeCraft

Für **jede Kombination aus Speicher und Softwarestand** (= eine Zeile im heutigen Freigabecockpit) wird ein Jira-Ticket erstellt; es verknüpft indirekt alle Outputs, indem die Links zu WMM und MIA angefügt sind. Die **Ticket-Granularität** ist **I-Stufe + Penthouse + Freigabelevel** (z. B. **26-7-490_D1_L1**). Kerninformationen je Ticket: **Due Date** = Freigabetag, **Target Start** = ATS-2, **Target End** = aktuell Freigabetag. **Alle Tickets einer I-Stufe** werden einem **Epic** untergeordnet. Updates werden in den Tickets ersichtlich gemacht. *Bezug zum aktuellen Stand:* Nicht umgesetzt — erfordert die Jira-Schnittstelle (§9.4, E3).

11.2.4 Freigabemuster

Eine **Word-Vorlage** wird je I-Stufe für alle Speicher erstellt, die eine **L2-Freigabe** erhalten haben, und teilbefüllt in SharePoint abgelegt. Automatisch ergänzter Inhalt: **Titel** (= Lead-I-Stufe), **Freigabekonfiguration** (= jede Speicherzeile aus App + WMM-Link) und eine **Auflistung der Muster je Penthouse**. Das Dokument wird den erzeugten Jira-Tickets angehängt. Zielbild: tägliches Update per Workflow mit **Freeze zwei Wochen vor** der Freigabe der jeweiligen I-Stufe. *Bezug zum aktuellen Stand:* Nicht umgesetzt (§7 R-7, E5).

11.2.5 MIA — technische Prüfung

Workflows sollen **MIA-Anlagen** durchführen und mit den korrekten Hardwarekonfigurationen aus den Stücklisten anreichern. HVS-Sachnummern werden nach MIA übertragen und prüfen dort, ob bereits eine Anlage vorliegt (falls ja → zuordnen, sonst → Neuanlage). MIA-Anlagen beschreiben eine **Level-Freigabe**; bei Speichern **im Verbund** referenzieren mehrere Speicher-Sachnummern denselben MIA-Link und werden in einem Freigabeobjekt zusammengefasst. Notwendige ID ist der **Name des Bewertungsprozesses**, den die App berechnet (z. B. **B6YJ0 D/D L2 26-7-480 ATS+3**). Freigaben werden wiederholt durchgeführt — fünf Stufen **RSTB, L1, L2, L3, L4**; bei der **ersten Freigabe ihrer Art je Speicher-SNR** wird eine MIA-Anlage erstellt. *Bezug zum aktuellen Stand:* Nicht umgesetzt; scheitert an der Microsoft×MIA-Anbindung. MIA-Badge als Platzhalter vorhanden (§9.5, E4). Die Berechnungslogik des Bewertungsprozess-Namens ist spezifiziert (§11.5).

11.3 Quellsysteme

11.3.1 IP3 (Langfristplanung, via CDH)

In IP3 werden je Hochvoltspeicher **Erstfreigabetermine je Musterphase** vergeben — die Langfristplanung der HVS-Freigaben. Die Termine liegen im **CDH** (Cloud Data Hub = BMW-Data-Lake), auf das die App per API zugreifen soll; Data Asset: „**IP3 Appointment Project Data Semantic**“. Jedem Speicher soll zudem ein **Fahrzeugprojekt** als übergeordneter Parent der Speicherprojekte zugewiesen sein (ebenfalls aus den IP3-Daten im CDH auslesbar).

11.3.2 Stücklisten (via CDH)

Stücklisten beschreiben das Set verbauter Sachnummern je HVS. Sie liegen im CDH, unterscheiden sich jedoch zwischen **Serie** und **Prototypenbau**:

- **Serie**: Data Asset „**PS-BOM-Semantic-Layer**“ auf Werksebene — je Werk eine HVS-ID, die sich in der Zusammensetzung der Sachnummern jedoch nicht unterscheidet; eine **Vorfilterung auf eindeutige Speicher-IDs** ist Grundvoraussetzung.
- **Prototyp**: Data Asset „**Everest Demand Data Iceberg Semantic**“.

Die Zuordnung von Sachnummern erfolgt über eine **zusammengesetzte HVS-ID** aus *Projektnummer + Musterphase + Bezeichnung* (z. B. **B6GE0-001_VS1_D1_Muster**). Jedem Speichermuster wird eine Stückliste zugeordnet und im WMM aktualisiert; CEs ordnen zudem **XCP-** und **XETK-Nummern** manuell zu. **Funktional relevante** Sachnummern gehen an das WMM, die **Gesamtheit aller** Sachnummern an MIA.

11.3.3 DEEP (Kurzfristplanung)

DEEP beschreibt die **Kurzfristplanung** der Freigabetermine und stützt sich auf eine Gesamtheit von Tickets aus **Jira CodeCraft, Jira ATC und SW-HRL**; die Tickets werden mit **Labels** versehen, um eine eindeutige Zuordnung zu ermöglichen. DEEP bildet die Zeitleiste und Termine der **Verbundfreigaben** ab (Verbund = die Gesamtheit des Antriebsstrangs eines Derivats) und liefert zusätzlich I-Stufen-Informationen. **Verbundfreigaben** werden auf **Penthouse-Level** erteilt: in der Regel werden alle HVS mit identischem Penthouse (PTH) gemeinsam freigegeben.

11.3.4 ESDH / Excel-Export

Quelle für I-Stufen-Folgen; Export-/Austauschformat.

11.4 Zielsysteme

- **MIA**: soll sich perspektivisch aus Jira-Tickets bedienen; Zielbild der App ist, jene Tickets lesen, auswerten, ggf. überschreiben oder neu anlegen zu können.
- **FUSI Confluence**: Dokumentations-/Ablageziel.

11.5 Schlüsselkonzepte & Berechnungslogiken

- **I-Stufe (Integration Level):** bezeichnet bei BMW den Software- und Hardware-Stand sämtlicher elektronischer Steuergeräte eines Fahrzeugs — welche Softwareversionen auf welchen Steuergeräten installiert sind und wie sie harmonisieren. Sie stellt sicher, dass alle Steuergeräte aufeinander abgestimmt und kompatibel sind. In der App treiben Reihenfolge und **farbliche Klassifizierung** der zusammengehörenden I-Stufen die Spaltenlogik (§4.1).
- **Freigabe-Level & Timeline:** fünf Stufen **RSTB · L1 · L2 · L3 · L4**. Orientierung im 8-Wochen-Fenster: **ATS = L1** , **ATS+2 = L2** , **SAB (= ATS+8) → L3** . Freigaben wiederholen sich; die erste je Speicher-SNR erzeugt eine MIA-Anlage.
- **Verbund vs. Verschränkung (Begriffsklärung):** „Verbund“ laut Anforderung = Zusammenfassen mehrerer HVS-Zeilen (gemeinsames Bearbeiten + Weitergabe an Folgesysteme, insb. MIA als ein Freigabeobjekt), erteilt auf Penthouse-Level. Die heutige App kennt die „**Verschränkung**“ (Bindung Softwarestand → Speicher, §4.3) als verwandtes, aber engeres Konzept; die echte Verbund-Gruppierung mit gemeinsamer Weitergabe ist Teil von E6F3 und noch offen.
- **HVS-ID / ZB_HVS-Nummer:** zusammengesetzte ID *Projektnummer + Musterphase + Bezeichnung*; im WMM ist die **ZB_HVS-Nummer** die eindeutige ID des Speicherprojekts/Zusammenbaus.
- **Jira-Ticket-Granularität & Felder:** **I-Stufe + Penthouse + Freigabelevel** ; Due = Freigabetag, Target Start = ATS–2, Target End = Freigabetag; Epic je I-Stufe.
- **MIA-Bewertungsprozess-Name:** wird durch die App berechnet, Schema **<Speicher> <D/D> <Level> <I-Stufe> <Offset>** (Beispiel **B6YJ0 D/D L2 26–7–480 ATS+3**).

11.6 Epic- & Feature-Backlog (E0–E7) mit Umsetzungsstand

Epic / Feature	Inhalt	Bezug / Status
E0F1	Umgebung, Security-Rollen, Dataverse-Tabellen/Modelle	Teilweise (Dataverse + App vorhanden; Security-Rollen offen)
E0F2	Custom Connectors (CDH, Jira), SharePoint-Integration	Offen (heute Dataflow „FGC_DATA“ über CSV statt Live-Connector)
E0F3	Logging, Audit, Telemetrie	Offen
E1F1	IP3-Termine (CDH) importieren & mappen (HVS↔Fahrzeugprojekt)	Teilweise (Tabellen vorhanden; kein Live-CDH-Import/Polling)
E1F2	Stücklisten Serie/Proto importieren & deduplizieren (eindeutige Speicher-IDs)	Offen (Dedup-Logik spezifiziert; nicht angebunden)
E1F3	DEEP-Daten lesen (Verbundtermine, Labels/Zuordnung)	Teilweise (Verbund-Daten via Snapshot sichtbar)
E1F4	Differenzlogik (SNR-Änderungen) & Hinweise	Teilweise (WMM-Delta-Mechanik, simuliert)
E2F1	„Nicht zugeordnete“ SNR, Quick-Assign, Bulk-Assign	Teilweise (SNR-Pflege im WMM-Drawer; „nicht zugeordnet“-Sicht simuliert)
E2F2	Workflow „Zusammenbau anlegen/aktualisieren“ (ID = ZB_HVS)	Offen (WMM-Schnittstelle)
E2F3	Update-Anzeige & „Überschreiben“-Aktion	Umgesetzt (UI, simuliert — WMM-Check + Übernehmen)
E3F1	Ticket-Erstellung je Speicher×SW-Stand (I-Stufe+PTH+Level) inkl. Links	Blockiert (Jira)
E3F2	Felder pflegen (Due Date, Target Start = ATS–2, Target End)	Blockiert (Jira)
E3F3	Updates/Transitions & Epic-Zuordnung je I-Stufe	Blockiert (Jira)
E4F1	Erstellen/Zuordnen MIA-Anlage (Bewertungsprozess-Name bilden)	Blockiert (MIA); Namenslogik spezifiziert (§11.5)
E4F2	Verbund-Handling (mehrere SNR → 1 MIA-Link)	Blockiert (MIA)
E4F3	Level-Wiederholungen (RSTB/L1/L2/L3/L4) verwalten	Offen
E5F1	Word-Vorlage generieren (Lead-I-Stufe, Konfiguration, Muster je PTH)	Offen
E5F2	Datei in SharePoint ablegen & Ticket-Anhänge setzen	Offen
E5F3	Täglicher Update-Workflow, Freeze –14 Tage vor I-Stufe	Offen
E6F1	Übersichtsgrid (Zeilen=HVS, Spalten=Wochen, Farblogik je I-Stufe)	Umgesetzt (§4.1)

Epic / Feature	Inhalt	Bezug / Status
E6F2	Live-Filter (I-Stufen, Wochen, Speicher, Fahrzeugprojekte)	Umgesetzt (Fahrzeugprojekt-Filter offen)
E6F3	Verbund bilden/bearbeiten & „Weitergabe an Folgesysteme“	Teilweise (Verschränkung umgesetzt; echte Verbund-Gruppierung + Weitergabe offen)
E7F1	KPI-Dashboards (Termintreue, Rework, Durchlaufzeiten)	Offen (Cockpit-Kennzahlen als Ansatz, §4.2)
E7F2	Integrations-Monitor (Flows, Fehler-Queues, Retry)	Offen